

UNIVERSIDAD MARIANO GÁLVEZ CENTRO UNIVERSITARIO AMATITLÁN
FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

Proyecta

Ruta mínima en el campus universitario

Brandon Eli Hernandez Morales

6691-26-15969

Oscar Javier Saravia Hernandez

6691-25-3835

Luis Fernando Hernandez Esquivel

6691-26-2522

Michael Eduardo Lemus Rodas

6691-26-6808

Javier Stiben Nimatuj Ajanel (coordinador)

6691-25-23517

Ing. Mario Alvarado

Guatemala, octubre 2026

Indice

Introducción 1

Teoría de grafos 2

- Aplicación en la vida real 2
- Tipos de grafos relevantes para el proyecto 3
- Conceptos fundamentales de grafos 3
  - Grafo 3
  - Nodo o vértice 4
  - Arista 4
  - Peso 5
  - Grafo no dirigido 5
  - Adyacencia y lista de adyacencia 6
  - Ejemplo con tres nodos 6
  - Nodo origen y nodo destino 7
  - Camino o ruta 7

Representación del grafo en el código 8

- Programación orientada a objetos 8
- Posición de los lugares en el mapa 8

El algoritmo Dijkstra 9

El algoritmo de Dijkstra en el proyecto 9

Ciclos while, for y condición if.....11

while: mantiene la búsqueda funcionando.....11

for: revisa las conexiones.....11

if: decide si encontramos una mejor ruta..... 12

Reconstrucción de la ruta..... 13

Mapa del campus 14

Introducción

En un campus universitario conviven estudiantes, docentes, personal administrativo y visitantes que se desplazan todos los días entre edificios, canchas, áreas de servicio y parqueos. Cada recorrido puede hacerse por distintos caminos, y no todos tienen la misma longitud. Elegir bien la ruta ahorra tiempo, evita recorridos innecesarios y es especialmente útil para quienes no conocen el lugar.

Este tipo de situación, en el que existen lugares relacionados entre sí por caminos con una distancia asociada, se puede representar de forma natural con un grafo. Los lugares pasan a ser puntos (nodos), los caminos pasan a ser líneas (aristas) y la distancia de cada camino pasa a ser el valor (peso) de la arista. Una vez que el campus está representado de esa manera, encontrar la ruta más corta entre dos lugares se convierte en un problema bien conocido de la informática: el problema del camino de costo mínimo.

Teoría de grafos

La teoría de grafos es una rama de las matemáticas discretas que estudia las relaciones y conexiones entre diferentes elementos. Estas relaciones se representan principalmente mediante nodos o vértices, que representan los elementos, y aristas, que representan las conexiones entre ellos. Formalmente, un grafo se define como un par G = (V, E), donde V es el conjunto de vértices y E es el conjunto de aristas.

Su origen se asocia con el problema de los puentes de Königsberg, que Leonhard Euler analizó en el siglo XVIII al preguntarse si era posible recorrer la ciudad cruzando cada puente exactamente una vez. Euler abstrajo el problema reemplazando las zonas de tierra por puntos y los puentes por líneas, y con ello dio origen a una forma de razonar que hoy se aplica en muchísimas áreas.

La teoría de grafos permite representar problemas de la vida real donde existen elementos relacionados entre sí y necesitamos analizar sus conexiones. Lo valioso de esta herramienta es que ignora los detalles que no importan (colores, formas, dibujos exactos) y se concentra en lo esencial: qué elementos existen y cómo están conectados.

Aplicación en la vida real

Los grafos aparecen en numerosas áreas. Algunos ejemplos representativos son los siguientes:

- **GPS y mapas digitales.** Las ubicaciones e intersecciones se representan como nodos, las carreteras como aristas y las distancias o los tiempos de recorrido como pesos. Así se pueden calcular rutas entre diferentes lugares.
- **Redes sociales.** Los usuarios son nodos y las relaciones entre ellos (amistad, seguimiento) son las conexiones.
- **Redes de computadoras.** Los dispositivos, servidores o routers son nodos y los enlaces de red son las aristas.
- **Logística y transporte.** Permiten analizar rutas de distribución, conexiones entre ciudades o recorridos de vehículos.

Nuestro proyecto aplica esta misma idea a un campus universitario, donde los lugares son los nodos, los caminos son las aristas y las distancias son los pesos.

Tipos de grafos relevantes para el proyecto

Existen varias clasificaciones de grafos. Para este trabajo interesan tres características que se combinan entre sí.

Para trabajar con un grafo en un programa hay que decidir cómo guardarlo en memoria. Las dos formas más comunes

Tipo	Descripción	En el proyecto
Dirigido	Cada arista tiene un sentido: ir de A a B no implica poder ir de B a A.	No se usa.
No dirigido	Las aristas se recorren en ambos sentidos.	Sí. Los caminos del campus se pueden recorrer de ida y de vuelta.
No ponderado	Todas las aristas son equivalentes; no tienen valor.	No se usa.
Ponderado	Cada arista tiene un peso (distancia, costo o tiempo).	Sí. El peso es la distancia en metros.
Conexo	Existe al menos un camino entre cualquier par de nodos.	Sí. Desde cualquier lugar se puede llegar a cualquier otro.

Conceptos fundamentales de grafos

Grafo

¿Qué es? Es un conjunto de nodos conectados mediante aristas.

¿Cómo se representa? Es todo el campus, incluyendo los lugares y los caminos que los conectan. En el programa lo representa la clase que guarda los edificios, las conexiones y la lista de adyacencia.

Nodo o vértice

¿Qué es? Es cada punto individual que forma parte del grafo.

¿Cómo se representa? Cada lugar del campus: Biblioteca, Rectoría, Cafetería, Cancha, Parqueo, etcétera. En el código los nodos se representan mediante la clase, que almacena el nombre, la posición en el mapa y el tipo de lugar:

```
static class Edificio {
    private final String nombre;
    private final int x, y;
    private final TipoLugar tipo;

    Edificio(String nombre, int x, int y, TipoLugar tipo) {
        this.nombre = nombre; this.x = x; this.y = y; this.tipo = tipo;
    }

    Edificio(String nombre, int x, int y) { this(nombre, x, y, TipoLugar.EDIFICIO); }

    String getNombre() { return nombre; }
    int getX() { return x; }
    int getY() { return y; }
    TipoLugar getTipo() { return tipo; }
}
```

Es decir, cada edificio puede almacenar su nombre, su posición y su tipo. El tipo permite distinguir los edificios comunes de la cancha y del parqueo, que se dibujan con otro color.

Arista

¿Qué es? Es la conexión entre dos nodos.

¿Cómo se representa? Son los caminos entre los lugares. Por ejemplo, Biblioteca ↔ Rectoría. En el código se utiliza la clase **Conexión**, donde **origen** y **destino** indican qué nodos están conectados:

```
static class Conexion {
    private final String origen, destino;
    private final int distancia;

    Conexion(String origen, String destino, int distancia) {
        this.origen = origen; this.destino = destino; this.distancia = distancia;
    }

    String getOrigen() { return origen; }
    String getDestino() { return destino; }
    int getDistancia() { return distancia; }

    boolean conecta(String a, String b) {
        return (origen.equals(a) && destino.equals(b)) || (origen.equals(b) && destino.equals(a));
    }
}
```

Peso

¿Qué es? Es el valor que se le asigna a una arista.

¿Cómo se representa? Es la distancia en metros de cada camino. Por ejemplo, Biblioteca ↔ Rectoría = 120 metros. En la clase el peso se almacena en el atributo:

```
private final int distancia;
```

Se usa un entero (**int**) porque las distancias del campus son números enteros de metros. Esto simplifica las comparaciones y evita errores de redondeo.

Grafo no dirigido

¿Qué es? Es un grafo donde una conexión puede recorrerse en ambos sentidos.

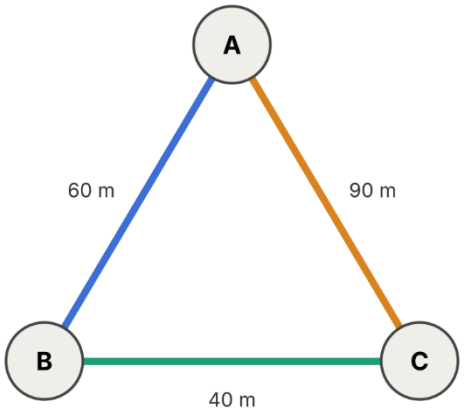
¿Cómo se representa? Si se puede ir de Biblioteca a Rectoría, también se puede ir de Rectoría a Biblioteca. En el programa, cada conexión se guarda dos veces en la lista de adyacencia, una por cada sentido, tal como se aprecia en el método:

```
void agregarConexion(String origen, String destino, int distancia) {
    conexiones.add(new Conexion(origen, destino, distancia));
    adyacencia.get(origen).add(new String[]{destino, String.valueOf(distancia)});
    adyacencia.get(destino).add(new String[]{origen, String.valueOf(distancia)});
}
```

Adyacencia y lista de adyacencia

¿Qué es? La adyacencia indica qué nodos están conectados directamente entre sí.

¿Cómo se representa? Por ejemplo, Biblioteca es adyacente a Rectoría y a Auditorio porque tiene caminos directos hacia ellos. En el programa se utiliza una lista de adyacencia:




```
private final Map<String, List<String[]>> adyacencia = new HashMap<>();
```

Esta estructura es un mapa que asocia el nombre de cada lugar con una lista de arreglos de texto. Cada arreglo contiene dos datos: el nombre del vecino y la distancia hacia él (guardada como texto). Así se conocen los vecinos directos de cada lugar y la distancia hacia ellos.

Ejemplo con tres nodos

La figura 1 muestra un grafo no dirigido muy pequeño con tres nodos (A, B y C) y tres aristas con sus pesos. La tabla 5 presenta su lista de adyacencia. Se puede observar que la conexión A–B de 60 m aparece tanto en la lista de A como en la de B: se guarda dos veces, una por cada dirección.

Nodo	Vecinos directos (distancia)
A	B (60 m), C (90 m)
B	A (60 m), C (40 m)
C	A (90 m), B (40 m)

Nodo origen y nodo destino

Nodo origen. Es el punto donde comienza el recorrido. En el programa es el lugar inicial que selecciona el usuario en la lista desplegable “Inicio”.

Nodo destino. Es el punto al que se quiere llegar. Es el lugar final seleccionado por el usuario en la lista desplegable “Destino”.

Camino o ruta

¿Qué es? Es una secuencia de nodos conectados que permite llegar desde el origen hasta el destino.

¿Cómo se representa? Por ejemplo: Biblioteca → Auditorio → Cancha → Parqueo. En el código, la ruta es una lista de nombres (List<String>) ordenada desde el origen hasta el destino, y se acompaña de su distancia total

Representación del grafo en el código

Programación orientada a objetos

En el proyecto se utiliza Programación Orientada a Objetos para representar de forma organizada los elementos del campus. Se crean clases que funcionan como moldes: a partir de cada clase se crean objetos con sus propios datos. Esta forma de trabajar separa responsabilidades: unas clases guardan información, otra ejecuta el algoritmo y otra dibuja el resultado.

Posición de los lugares en el mapa

Para dibujar el campus, cada edificio necesita una posición. En la versión 2D, esa posición son las coordenadas `x` (horizontal) e `y` (vertical) que recibe el constructor de **Edificio**. La documentación del proyecto describe también una clase auxiliar, **Posicion2D**, que guarda estas coordenadas en la versión con JavaFX, independientes de la posición tridimensional del edificio.

Este diseño permite que el mismo grafo se dibuje de varias maneras (plano 2D, vista 3D) sin cambiar la lógica del algoritmo, porque Dijkstra solo necesita los nombres de los lugares y las distancias, no sus coordenadas.

El algoritmo Dijkstra

El algoritmo de Dijkstra, propuesto por el científico holandés Edsger W. Dijkstra en 1959, calcula los caminos de menor costo desde un nodo origen hacia los demás nodos de un grafo cuyas aristas tienen pesos no negativos. Es un algoritmo voraz (greedy): en cada paso toma la mejor decisión local, que es visitar el nodo no revisado con menor distancia conocida, y se puede demostrar que esa decisión nunca impide encontrar la solución óptima.

La idea central puede resumirse en cuatro ideas:

- Se mantiene, para cada nodo, la mejor distancia conocida desde el origen. Al comenzar, el origen vale 0 y los demás nodos valen «infinito», porque todavía no se sabe cómo llegar a ellos.
- En cada paso se elige el nodo no visitado de menor distancia. Esa distancia ya es definitiva, porque cualquier otro camino hacia ese nodo tendría que pasar por nodos aún más lejanos.
- Desde ese nodo se revisan sus vecinos. Si llegar a un vecino pasando por el nodo actual cuesta menos que la distancia que ya se tenía, se actualiza esa distancia y se anota por dónde se llegó.
- El proceso se repite hasta que no quedan nodos por revisar. Luego se recorre el camino hacia atrás, desde el destino, usando lo anotado.

El algoritmo de Dijkstra en el proyecto

Dijkstra explora las conexiones del grafo y va comparando las distancias acumuladas. Cada vez que encuentra una forma más corta de llegar a un nodo, actualiza esa distancia. Al terminar, reconstruye la ruta que tiene la menor distancia desde el punto inicial hasta el destino. El programa no compara únicamente caminos individuales: va sumando los pesos de las aristas recorridas para finalmente mostrar la ruta más corta posible.

Paso	Qué ocurre	Líneas del código
1	Se asigna distancia 0 al inicio e «infinita» a los demás.	81–84
2	Se prepara la cola de prioridad y el conjunto de visitados.	86–88
3	Mientras haya pendientes, se toma el más cercano no visitado.	90–93
4	Se revisan sus vecinos y, si hay un camino mejor, se actualiza.	94–103
5	Se verifica si el destino fue alcanzado.	106
6	Se reconstruye la ruta hacia atrás y se devuelve el resultado.	108–114

```
81          Map<String, Integer> distancias = new HashMap<>();
82          Map<String, String> previos = new HashMap<>();
83          for (String nombre : edificios.keySet()) distancias.put(nombre, Integer.MAX_VALUE);
84          distancias.put(inicio, 0);
85
86          PriorityQueue<String> pendientes = new PriorityQueue<>(Comparator.comparingInt(distancias::get));
87          pendientes.add(inicio);
88          Set<String> visitados = new HashSet<>();
89
90          while (!pendientes.isEmpty()) {
91              String actual = pendientes.poll();
92              if (visitados.contains(actual)) continue;
93              visitados.add(actual);
94              for (String[] vecino : adyacencia.get(actual)) {
95                  String nombreVecino = vecino[0];
96                  int peso = Integer.parseInt(vecino[1]);
97                  int nuevaDistancia = distancias.get(actual) + peso;
98                  if (nuevaDistancia < distancias.get(nombreVecino)) {
99                      distancias.put(nombreVecino, nuevaDistancia);
100                     previos.put(nombreVecino, actual);
101                     pendientes.add(nombreVecino);
102                 }
103             }
104         }
```

Ciclos while, for y condición if

Dijkstra utiliza principalmente ciclos `while` y `for` para recorrer los nodos y sus conexiones, y una condición `if` para comparar las distancias y actualizar la ruta cuando encuentra una opción más corta.

```
while (!pendientes.isEmpty()) {
    String actual = pendientes.poll();
    if (visitados.contains(actual)) continue;
    visitados.add(actual);
    for (String[] vecino : adyacencia.get(actual)) {
        String nombreVecino = vecino[0];
        int peso = Integer.parseInt(vecino[1]);
        int nuevaDistancia = distancias.get(actual) + peso;
        if (nuevaDistancia < distancias.get(nombreVecino)) {
            distancias.put(nombreVecino, nuevaDistancia);
            previos.put(nombreVecino, actual);
            pendientes.add(nombreVecino);
        }
    }
}
```

while: mantiene la búsqueda funcionando

El while hace que el algoritmo siga buscando mientras todavía existan nodos por revisar. En cada repetición toma el nodo que tiene la menor distancia conocida hasta ese momento: pendientes.poll() obtiene de la cola de prioridad el nodo que corresponde revisar. Si ese nodo ya había sido visitado (puede aparecer repetido en la cola), la instrucción continue lo salta. En palabras simples: «mientras queden lugares por revisar, continúa buscando la ruta más corta».

for: revisa las conexiones

Una vez que el while selecciona un edificio, el for se encarga de recorrer todos los edificios que están conectados directamente con él. Por ejemplo, si se está en Biblioteca y esta tiene caminos hacia Rectoría y Auditorio, el for revisará ambos caminos y sus respectivas distancias. adyacencia.get(actual) obtiene las conexiones directas del nodo que se está revisando; de cada arreglo se extraen el nombre del vecino (posición 0) y el peso (posición 1, convertido a entero con

Integer.parseInt). En palabras simples: «revisa uno por uno todos los caminos disponibles desde el lugar donde estamos».

if: decide si encontramos una mejor ruta

Después de calcular cuánto costaría llegar a un edificio por un determinado camino, el if compara esa distancia con la mejor distancia que ya se tenía guardada. Primero se calcula nuevaDistancia = distancias.get(actual) + peso, es decir, la distancia que se lleva más la distancia de la nueva conexión. Después se compara: si la nueva distancia es menor, se actualiza; si es mayor, simplemente se mantiene la distancia anterior. Cuando se actualiza, además se anota desde dónde se llegó (previos.put) y se agrega el vecino a la cola (pendientes.add). En palabras simples: compara las rutas y, si encuentra una más corta, reemplaza la anterior.

Estructura	Función	Idea en una frase
`while`	Mantiene la búsqueda	Mientras queden lugares por revisar, continúa.
`for`	Revisa las conexiones	Mira uno por uno los caminos disponibles.
`if`	Compara y decide	Si hay una ruta más corta, reemplaza la anterior.

Reconstrucción de la ruta

Después de calcular las distancias, el programa necesita determinar por cuáles lugares debe pasar. Para esto utiliza el mapa previos. Durante Dijkstra se guarda, cada vez que se mejora una distancia, previos.put(nombreVecino, actual), lo que permite recordar desde qué nodo se llegó. Después se comienza desde el destino y se retrocede hasta el origen:

```
        if (distancias.get(fin) == Integer.MAX_VALUE) return new ResultadoRuta(false, new ArrayList<>(), 0);

        List<String> ruta = new ArrayList<>();
        String paso = fin;
        while (paso != null) {
            ruta.add(0, paso);
            paso = previos.get(paso);
        }
        return new ResultadoRuta(true, ruta, distancias.get(fin));
    }
}
```

Si la distancia del destino sigue siendo «infinita», significa que no hay camino y se devuelve un resultado con existe = false. En caso contrario, el ciclo while parte del destino (paso = fin) y, mientras `paso` no sea nulo, inserta ese lugar al principio de la lista con ruta.add(0, paso) y retrocede con previos.get(paso). Como el origen no tiene previo, previos.get devuelve null y el ciclo termina. Al insertar siempre al inicio, la lista queda ordenada del origen al destino. De esta manera se obtiene una ruta como Biblioteca → Auditorio → Cancha → Parqueo.

Mapa del campus

La figura 2 muestra el grafo completo tal como lo dibuja el programa: cada lugar es un rectángulo redondeado, cada camino es una línea y sobre ella aparece su distancia. La cancha se dibuja en verde oscuro y el parqueo en gris, para diferenciarlos de los edificios.

